第三次实验报告
基于 DeepSeek 的 RAG 信息检索系统

姓名：唐正浩
学号：2454330
组队成员：方鹭洋 2454331
日期：2026 年 4 月 24 日

目录

1 实验背景与目标 **2**

2 RAG 原理与系统流程 **2**

2.1 RAG 原理简述 2

2.2 系统流程框图 2

3 系统实现与技术选型 **3**

3.1 知识库与检索实现 3

3.2 生成模块与 Prompt 设计 5

3.3 拒答策略与工程化处理 7

3.4 技术选型说明 7

4 实验设置 **8**

4.1 运行环境与关键参数 8

4.2 问题集设计 8

5 实验结果与分析 **9**

5.1 总体结果 9

5.2 典型案例分析 10

5.2.1 案例一：技术框架问题（q3） 10

5.2.2 案例二：关键词抽取问题（q4） 11
5.2.3 案例三：研究目标与应用场景（q1, q2） 12
5.3 失败案例分析 12

6 交互界面与运行截图 **13**

7 上传参考资料与自动更新知识库 **13**
7.1 实验案例：上传测试文档并立即检索 14

8 问题、排查与改进 **17**
8.1 实验过程中遇到的问题 17
8.2 后续改进方向 17

9 结论与心得 **17**

参考资料 **18**

1 实验背景与目标

本次实验要求在已有 DeepSeek 调用能力的基础上，引入 Retrieval-Augmented Generation（RAG）技术，构建一个可检索、可生成、可追溯的问答系统。系统不仅需要完成对用户问题的向量检索和最终答案生成，还需要支持 `llm_only`、`retrieval_only` 与 `rag` 三种模式的对比，并输出结构化实验结果和可演示的交互界面。

本次实验由我和同伴方鹭洋协作完成。整体上，方鹭洋负责检索前半链路，包括文档收集、文本切分、embedding、向量库构建与检索逻辑；我负责生成后半链路，包括 DeepSeek 调用、Prompt 设计、三模式对比实验、Streamlit 界面和文档整合。

2 RAG 原理与系统流程

2.1 RAG 原理简述

RAG 的核心思想是“先检索，再生成”。与只依赖大模型内部参数记忆的传统问答方式相比，RAG 在回答问题前会先从外部知识库中找到与问题最相关的文档片段，再把这些片段作为约束上下文送入大模型。这样做的直接好处在于，回答会更贴近知识库中的真实内容，整体准确性更高；与此同时，回答还可以附带来源引用，因而具有更好的可追溯性，也更容易抑制幻觉。

在本实验中，我实现的系统支持三种模式。`llm_only` 不检索，直接询问 DeepSeek；`retrieval_only` 只展示检索片段，不做生成；`rag` 则先检索，再基于证据生成回答。通过把三种模式放在同一问题集上对比，我可以更清楚地观察“只靠模型”“只靠检索”和“检索增强生成”之间的差异。

2.2 系统流程框图

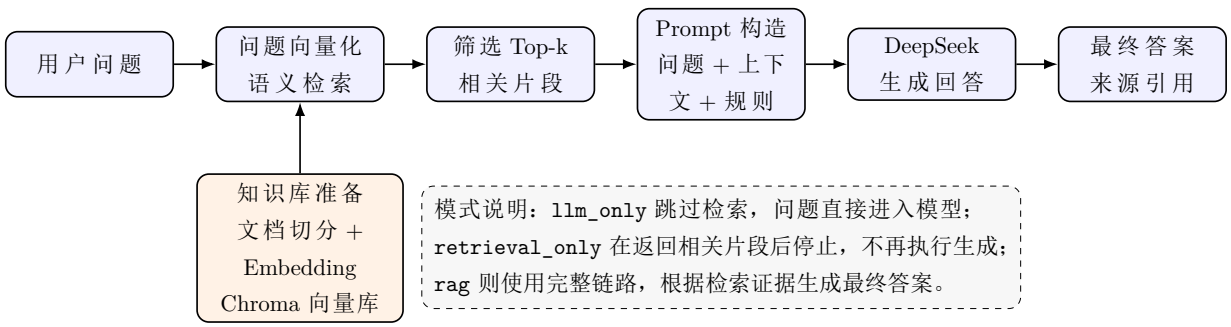


图 1: RAG 问答系统整体流程框图

从工程整合结果看，整个系统被拆分为四层。数据与检索层负责向量库访问和检索结果标准化，生成层负责 DeepSeek 客户端创建、消息构造与回答生成，实验层负责三模式

批量对比与结果落盘，展示层则负责 Streamlit 交互界面与演示。这样的分层方式使检索逻辑、生成逻辑和展示逻辑相互独立，便于我后续继续替换检索后端或扩展实验设置。

3 系统实现与技术选型

3.1 知识库与检索实现

结合该 notebook、4 篇原始 PDF 文档以及已经交付的 Chroma 数据库，可以将方鹭洋负责的检索侧实现概括为一条完整的 ETL 流程：从文档目录批量读取 PDF，执行文本清洗与切分，完成向量化并持久化到 Chroma，最后再通过检索函数输出可直接交给大模型的上下文结果。更具体地说，当前知识库并不是泛领域资料，而是围绕两个相邻主题组织的：前三篇文档主要集中在“大模型应用”和“大学生心理咨询”的交叉场景，最后一篇文档则转向“博物馆新媒体与青少年科普”。因此，这个系统当前更适合回答大学生心理咨询、大模型落地应用和知识增强问答相关的问题，而不应被理解为开放领域的通用知识库。图 2 展示了本次建库时实际纳入的 4 篇 PDF。



图 2: 知识库原始文档列表截图

在文档读取阶段，notebook 中把“读取 PDF + 清洗 + 切分”封装成一个主流程函数，并在 Google Drive 中指定统一文档目录。其核心配置可以概括为：

```
process_and_split_documents_with_ocr(directory_path)
DOCS_DIR=<docs-directory>
PyPDFLoader(file_path,extract_images=True)
```

同时，notebook 的环境依赖还安装了 `rapidocr-onnxruntime`，这说明检索侧确实采用了“文本层优先 + 图像页 OCR 回退”的路线来兼容原生 PDF 与可能存在的扫描页或嵌套图片。这种设计比较适合中文学术资料，因为文档中往往同时包含规范正文、复杂版式和局部图像文本，单一路径提取容易遗漏信息。

在文本清洗阶段，notebook 中额外实现了 `clean_academic_text(text)`。该函数会通过正则表达式剔除“中国知网”水印、JOURNAL OF ... 年份这类期刊干扰信息，以及“第 X 卷第 X 期”之类的卷期号文本；随后再压缩多余空格和过密换行。清洗完成后，代码没

有直接丢弃元数据，而是重新封装为 `Document(page_content=cleaned_text,metadata=page.metadata)`，从而保留每页原始的 `source` 与 `page` 信息。这一点对本次作业尤其重要，因为在后续生成答案时需要可追溯的来源，而不是只保留纯文本内容。

在文本切分阶段，代码中使用的是 `RecursiveCharacterTextSplitter`，并明确设置 **chunk size = 400**、**chunk overlap = 80**，分隔符为双换行、句号、感叹号、问号、分号、单换行、逗号和空格。切分逻辑由 `text_splitter.split_documents(cleaned_docs)` 执行，生成结果聚合到 `chunks_ready_for_db`。这一组参数与课程建议的“300–500 字符、50–100 字符 overlap”完全一致，同时也体现了中文文档切分时“先保段落、再保句子、最后必要时再细分”的设计取向。对当前知识库来说，400 字能够较稳定地容纳一个相对完整的论点或摘要片段，而 80 字重叠可以在跨句、跨段甚至跨页边界时补回一部分上下文连续性。

在向量化与存储阶段，notebook 采用 BGE 中文嵌入模型，并把切片结果直接写入 Chroma 持久化目录。核心调用如下：

```
HuggingFaceEmbeddings(model_name="BAAI/bge-small-zh")
Chroma.from_documents(documents=chunks_ready_for_db,...)
persist_directory=<chroma-db-directory>
```

在提交版本中，实际使用的是已经附带的 Chroma 数据库副本：

```
assets/vector_store/teammate_chroma_db
```

此外，notebook 末尾还通过 `sqlite3` 直接进入 `chroma.sqlite3` 做抽查，说明方鹭洋不仅完成了建库，而且实际检查过切片与元数据已经成功写入持久化数据库。图 3 展示了对 `chroma.sqlite3` 执行 `.tables` 抽查时看到的结果。这里至少可以确认当前提交版本中的向量库不是空目录，而是已经包含 `collections`、`embeddings`、`segments` 等 Chroma 核心表，因此知识库数据确实已经完成持久化。

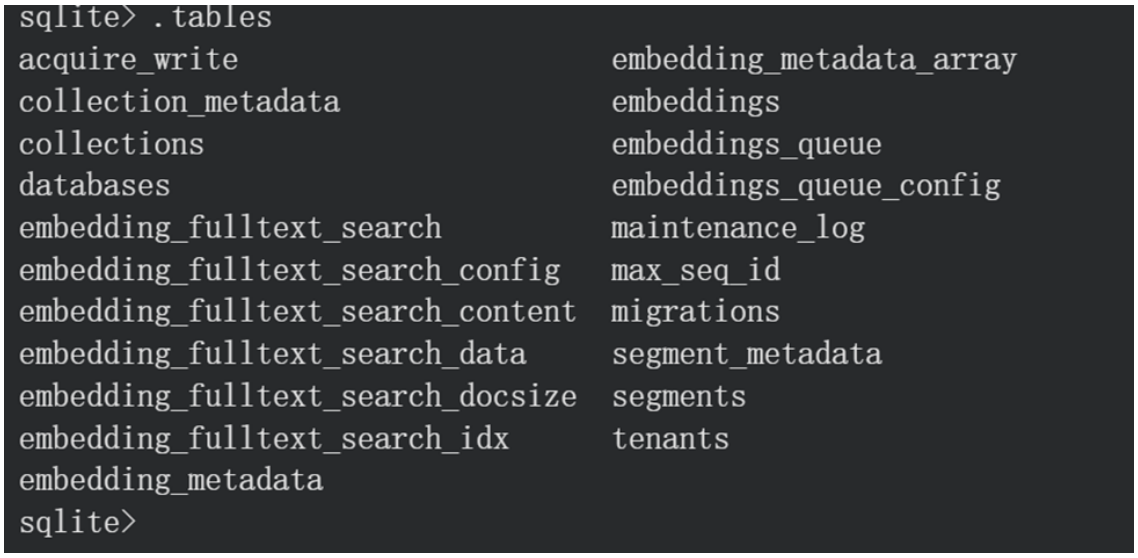


图 3: Chroma 持久化数据库表结构抽查截图

在检索阶段，notebook 中实现了一个直接面向下游 Prompt 的 Markdown 检索接口，其关键约束如下：

```
get_retrieved_context_md(query,k=3,score_threshold=0.8)
similarity_search_with_score(query,k=k)
score>0.8
```

该函数先执行 Top-K 检索，再按阈值过滤低相关片段，并将保留下来的结果格式化成 Markdown 字符串，内容包括资料片段编号、来源文件名、页码、距离分值和正文文本。这里的 `score` 不是“越大越好”的相似度分，而是 BGE 路线下的距离分，数值越小代表语义越接近。也正因为方鹭洋在 notebook 中直接交付的是 Markdown 上下文字符串，所以我在工程侧又补了一层统一适配：当前前端、CLI 和实验脚本并不是直接消费这个 Markdown 字符串，而是通过 `src/teammate_chroma_retriever.py` 直接连接同一个持久化 Chroma 数据库，把检索结果标准化为 `list[dict]`。这样既保留了方鹭洋原始 notebook 的实现思路，也让后续的 Prompt 构造、实验记录和可视化前端更容易复用统一数据结构。

3.2 生成模块与 Prompt 设计

我负责的生成侧主要由四个模块组成：

```
src/deepseek_service.py
src/prompt_builder.py
src/main.py
frontend/app.py
```

其中，src/deepseek_service.py 负责统一封装 DeepSeek 客户端创建与聊天调用，src/prompt_builder.py 负责构造 llm_only 与 rag 两类消息，src/main.py 提供单条问答的 CLI 入口，frontend/app.py 则提供 Streamlit 可视化界面。这样的划分使核心逻辑既能被脚本复用，也能被前端复用。

Prompt 设计采用三级策略档位。**strict** 强调证据不足即拒答，优先控制幻觉风险；**balanced** 兼顾稳健性与可读性，被设为默认档位；**concise** 则更强调回答简短，适合前端快速展示。在 RAG 模式下，系统要求模型只能基于参考内容作答，禁止补充外部常识；如果证据不足，就必须输出固定拒答句“根据现有资料无法回答”；如果能够回答，则需要按“回答 + 来源编号”的格式输出，并且来源编号必须来自实际参考内容，而不能由模型自行编造。

由于本次正式实验实际使用的是 balanced 档位，因此代码中真正送入模型的 Prompt 模板原文如下。其中，<question> 和 <context-block> 会在运行时被替换为具体问题和检索片段。

llm_only 模式 Prompt

system
你是精确的问答助手。当前回答风格：2-4 句回答，并附来源编号。

user
<question>

rag 模式 Prompt

system
你是一个 RAG 问答助手。策略档位：balanced；在可回答前提下保持简洁和可读性。

user
请严格遵守以下规则：
1) 你只能基于【参考内容】回答，禁止补充任何外部常识。
2) 若证据不足，必须且只能输出：根据现有资料无法回答
3) 若可回答，输出格式固定为：
回答：< 简洁结论 >
来源：< 按 [1] [2] 编号列出 >
4) 不允许输出未在参考内容中出现的来源编号。
5) 当前风格：2-4 句回答，并附来源编号。
6) 风险控制：仅使用参考内容，不补充外部事实。

【参考内容】
<context-block>

【用户问题】

```
<question>
```

3.3 拒答策略与工程化处理

为了进一步降低幻觉，我实现了两层拒答控制。一层是 Prompt 级约束，也就是在提示词中明确要求模型在缺乏证据时拒答；另一层是程序级约束，即当检索为空、检索片段无有效文本或证据明显不足时，直接输出拒答句，而不是继续让模型自由生成。我还补充了一项修复：模型偶尔会返回类似“回答：根据现有资料无法回答。来源：无”的包装式拒答。为保持实验口径一致，我把这类输出统一归一化为唯一标准文本“根据现有资料无法回答”，该逻辑已经接入 CLI、实验脚本和 Streamlit 界面。在正式实验实际发送给 DeepSeek 的 Prompt 中，与拒答直接相关的原文约束可以单独摘录为下面这几行。它们都来自 `src/prompt_builder.py` 中 `build_rag_messages(...)` 的真实用户提示词，而不是我在报告里的事后概括：

user
请严格遵守以下规则：
1) 你只能基于【参考内容】回答，禁止补充任何外部常识。
2) 若证据不足，必须且只能输出：根据现有资料无法回答
4) 不允许输出未在参考内容中出现的来源编号。
6) 风险控制：仅使用参考内容，不补充外部事实。

也就是说，Prompt 层的拒答并不是一句宽泛的“尽量保守回答”，而是把“只能基于参考内容”“证据不足时只能输出固定拒答句”“不能编造来源编号”三件事同时写成了硬约束。程序层再进一步用 `should_force_refusal(...)` 和 `normalize_rag_output(...)` 做兜底：前者在检索结果为空或无有效文本时直接拒答，后者则把“回答：根据现有资料无法回答。来源：无”这类包装式输出版式归一化为唯一标准文本，以保证 CLI、批量实验和前端页面的口径一致。

3.4 技术选型说明

我在本次实验中的主要技术选型如表 1 所示。

表 1: 系统技术选型

组件	选型与说明
大模型调用	DeepSeek API，当前使用 `deepseek-chat`
Embedding 模型	`BAAI/bge-small-zh`，适合中文语义检索
向量数据库	Chroma，支持本地持久化和相似度搜索
检索后端适配	`none/stub/teammate_chroma` 三种后端统一接入
交互界面	Streamlit，便于快速搭建实验演示页面
实验输出	Markdown + JSON，便于回溯和在报告中引用
测试工具	Pytest，用于基础单元测试

4 实验设置

4.1 运行环境与关键参数

我在本次正式实验中采用 `teammate_chroma` 作为检索后端，Prompt 档位为 `balanced`。所使用的 embedding 模型为 `BAAI/bge-small-zh`，检索参数固定为 `top_k=3` 与 `score_threshold=0.8`，整体设置与方鹭洋的建库和检索说明保持一致。对应向量库目录为：

```
assets/vector_store/teammate_chroma_db
```

本次实验的正式结果文件如下，后文中的结果分析均基于这次复跑结果展开：

```
assets/outputs/compare_20260424_092457.json
assets/outputs/compare_20260424_092457.md
```

4.2 问题集设计

为了让问题与当前知识库尽可能匹配，本次默认问集扩展为 5 个问题，其中前 4 个问题用于展示 RAG 在已命中文档上的回答能力，第 5 个问题作为失败案例，用于观察检索偏移时的系统表现。问题集见表 2。

表 2: 实验问题集

题号	问题	设计目的
q1	What problem does the college mental health counseling paper aim to solve?	验证 RAG 能否提炼论文目标
q2	What roles can large language models play in college mental health counseling?	验证 RAG 能否概括应用场景
q3	What kind of technical framework does the paper propose for college counseling?	验证 RAG 能否基于摘要生成结构化回答
q4	What keywords are listed in the college mental health counseling paper?	验证 RAG 能否抽取明确事实
q5	How can museums use AI-powered new media for science education?	作为失败案例，观察检索偏移与 RAG 拒答

5 实验结果与分析

5.1 总体结果

我复跑后三种模式均成功运行。统计得到的平均延迟如表 3 所示。

表 3: 三模式平均延迟对比

模式	平均延迟
`llm_only`	2369 ms
`retrieval_only`	144 ms
`rag`	1379 ms

从结果看，`retrieval_only` 速度最快，但输出只是原始片段，可读性和可用性都较弱；`llm_only` 语言表达最流畅，却更容易在缺乏证据时进行主观补全；`rag` 的延迟介于两者之间，但同时兼顾了自然语言可读性和证据约束，因此更符合课程作业对“准确、可溯源、可分析”的要求。

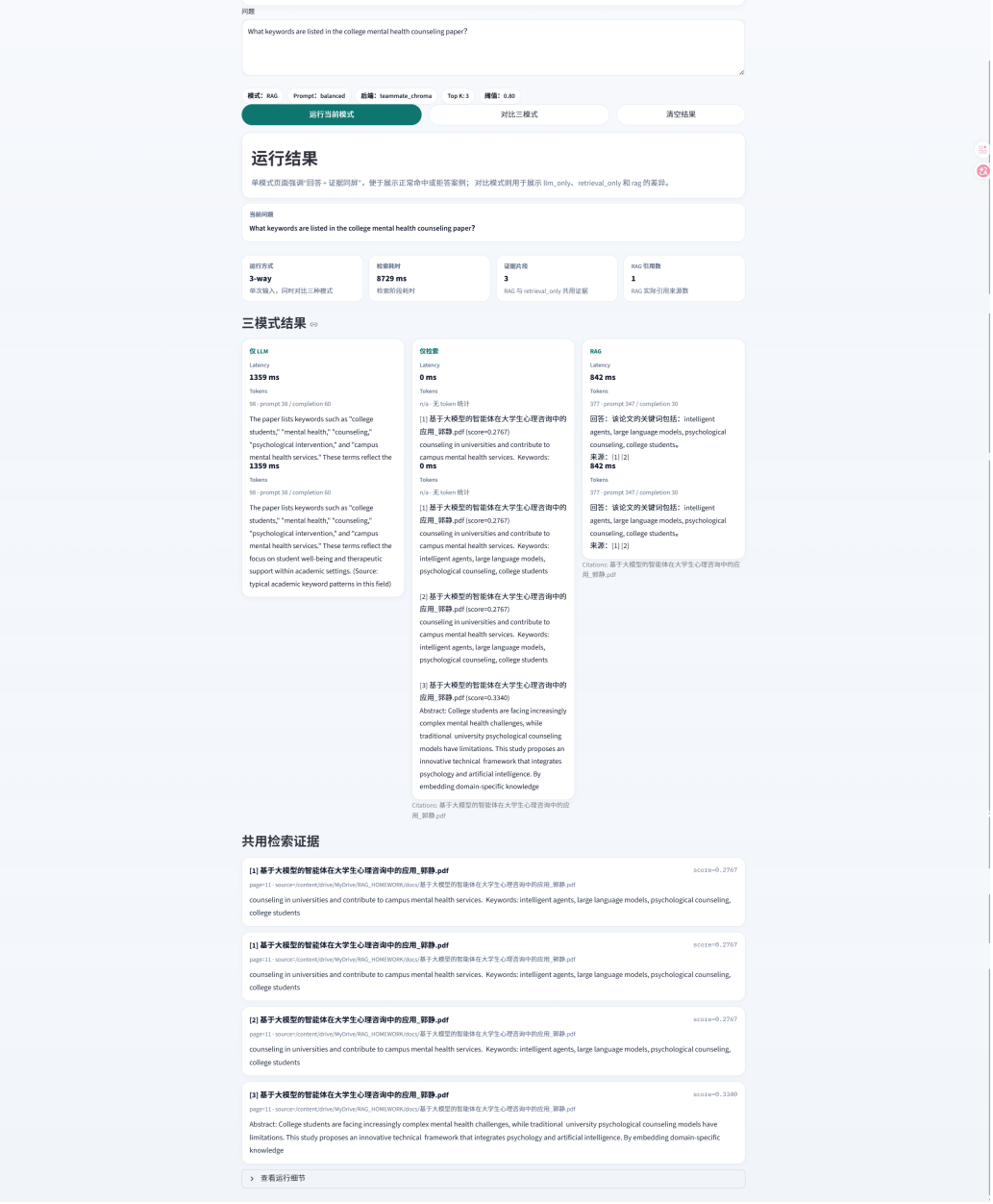


图 4: q4 问题的三模式对比页面

5.2 典型案例分析

5.2.1 案例一：技术框架问题（q3）

问题：*What kind of technical framework does the paper propose for college counseling?*

在这个问题上，`llm_only` 给出了“基于 CNN 的深度学习框架”这类回答，但这一内容并不来自当前知识库，属于典型幻觉；`retrieval_only` 则直接返回了摘要中的原始文本，其中明确提到该研究提出了整合心理学与人工智能的创新技术框架；`rag` 在此基础上生成“该论文提出了一种整合心理学与人工智能的创新技术框架，通过嵌入领域特定知识来改进大学心理咨询服务”，整体与检索证据保持一致。

这个例子表明，RAG 的优势不只是“会回答”，而是“能在证据基础上回答”。

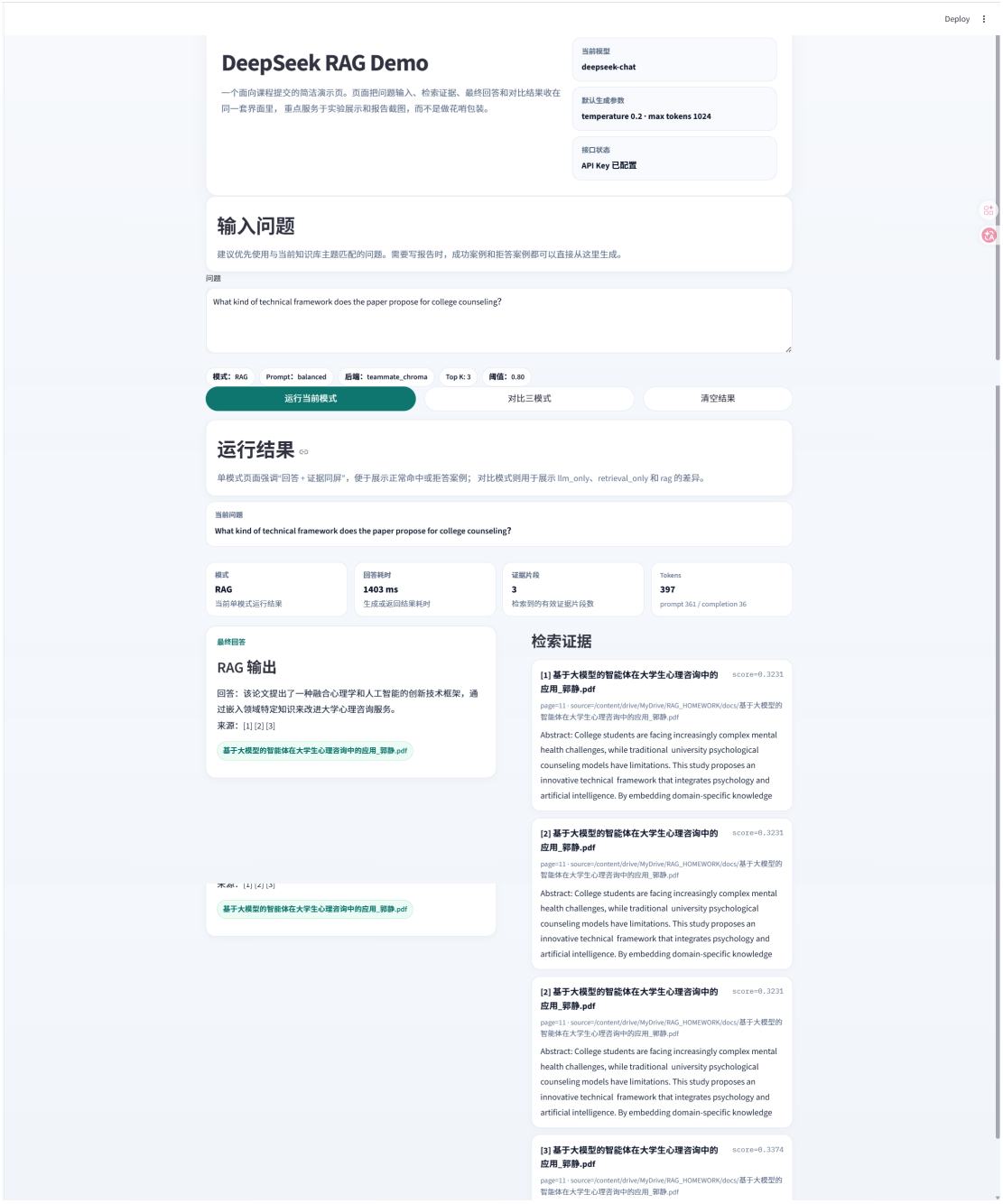


图 5: q3 问题下的正常检索与引用回答页面

5.2.2 案例二：关键词抽取问题（q4）

问题：*What keywords are listed in the college mental health counseling paper?*

在这个问题上，`llm_only` 倾向于根据一般论文经验猜测关键词，如 mental health、intervention strategies 等，但这些并非当前文档中给出的原始关键词；`retrieval_only` 返回了带有 `Keywords:` 的原文片段；`rag` 则正确抽取出 *intelligent agents, large language models, psychological counseling, college students*。

这个案例说明，面对“事实抽取类问题”，RAG 比纯 LLM 更容易给出可验证、可追溯的结果。

5.2.3 案例三：研究目标与应用场景（q1, q2）

对于论文目标和大语言模型作用这两类问题，`llm_only` 虽然可以给出通顺回答，但这些回答并不能证明来源于当前知识库；而 `rag` 可以根据检索到的摘要片段，将关键内容压缩为更短、更稳健的答案，并附带来源编号，适合我在实际界面中展示。

5.3 失败案例分析

失败案例选取 q5：

How can museums use AI-powered new media for science education?

q5 的检索结果仍以“大学生心理咨询”相关片段为主，说明当前检索阶段存在明显的主题偏移。

在该问题上，`llm_only` 给出了流畅且看似合理的回答，但没有证据支撑；`retrieval_only` 直接暴露了检索错误，返回了不相关片段；`rag` 最终选择输出“根据现有资料无法回答”，从而避免了错误信息进一步扩散。

这一结果说明，RAG 的意义不仅在于提升回答质量，也在于当检索证据不可靠时，通过拒答机制主动降低幻觉风险。

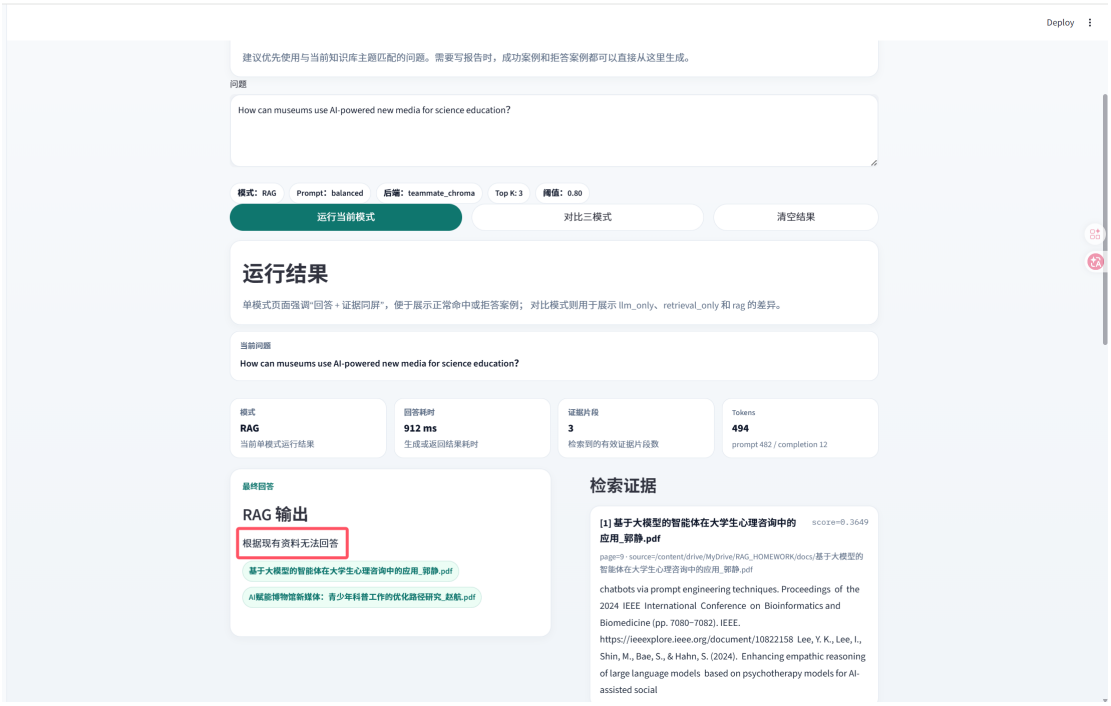


图 6: q5 失败案例中的拒答与检索证据页面

6 交互界面与运行截图

我实现了一个基于 Streamlit 的可视化界面，启动命令如下：

```
powershell-ExecutionPolicyBypass-Filescripts/run_ui.ps1
```

界面支持 `llm_only/retrieval_only/rag` 三种模式切换，也支持 `strict/balanced/concise` 三种 Prompt 档位切换，同时还能在 `none/stub/teammate_chroma` 三种检索后端之间切换，并调整 `top_k`、`embedding_model` 和 `score_threshold` 等关键参数。因此，这个界面不仅能满足课程演示要求，也方便我在调参时快速比较不同设置下的系统表现。图 7 展示了首页与参数区，图 4、图 5 和图 6 则分别对应三模式对比、正常引用回答和失败案例拒答页面。

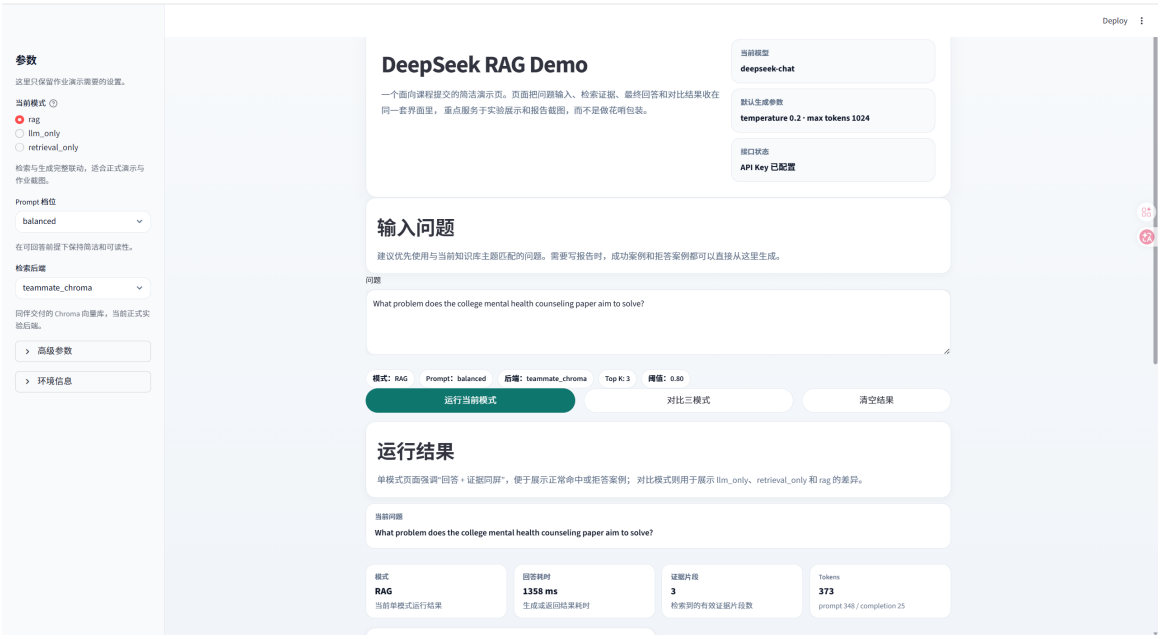


图 7: Streamlit 首页与参数设置界面

7 上传参考资料与自动更新知识库

在完成基础问答、三模式对比和拒答控制之后，我又进一步补充了一个更贴近真实使用场景的前端能力：允许用户在界面中直接上传新的参考资料，并在上传成功后自动把该文件增量写入当前 Chroma 向量库，而不是只把文件复制到资料目录中等待手动重建。这样一来，系统就不再只是一个“依赖固定知识库”的演示原型，而具备了基本的在线扩展能力。

这一功能在工程上被拆成前后两层。前端侧仍然由 `frontend/app.py` 提供上传入口，负责接收文件、展示当前保存目录和返回写库状态；后端侧则新增了 `src/document_ingest.`

py，把文档加载、文本清洗、切分、embedding 与 Chroma 增量写入封装成可复用模块。具体流程是：先根据扩展名识别文档类型，再对 PDF、Markdown、TXT 和 DOCX 分别提取文本；随后复用与原知识库一致的清洗规则和 400/80 切分参数生成切片；最后使用 BAAI/bge-small-zh 生成向量，并把切片写入当前使用的 assets/vector_store/teammate_chroma_db。如果用户上传的是同名文件，系统会先按 source 元数据删除旧切片，再写入新切片，以避免新旧内容同时保留而污染检索结果。

从交互效果上看，上传成功后页面会直接返回“已写入几份文档、几个切片以及实际更新的向量库目录”，用户无需离开当前界面，也不需要额外执行建库脚本。为了验证这一点，我使用一份带有唯一事实字段的测试文档进行实验：上传完成后，系统能够在同一个 Chroma 库中检索到该新文件，并在 RAG 模式下基于新证据给出正确答案。这说明当前实现已经能够把“上传资料”和“知识库更新”连成一个闭环，而不只是停留在文件层面的保存。

7.1 实验案例：上传测试文档并立即检索

为了对这项功能做最直接的验证，我额外构造了一份名为“测试文档 1.txt”的纯文本资料，其内容包含几个不会在原始知识库中自然出现的唯一事实字段，例如“项目代号：Aster-7”和“系统负责人：Li Ming”。这样设计的目的，是避免模型从原有论文资料中误命中相似表达，从而更清楚地判断新文件是否真的已经被写入当前向量库。

完成上传后，前端在左侧直接返回“已覆盖同名资料并刷新向量库”以及“已写入 1 份文档、1 个切片”的状态信息，这说明这份短文本资料并没有停留在文件目录层面，而是已经完成了切分和增量写库。随后，我立刻在同一页面输入问题：

南开大学 RAG 项目的负责人是谁？

在这次实验中，系统最终给出的回答为“根据参考内容，南开大学 RAG 项目的负责人是 Li Ming。”，成功命中上传文档中的唯一字段“Li Ming”，同时右侧检索证据也显示来源为“测试文档 1.txt”的片段。如果这一问答链路能够成立，就说明“上传文件 $\rightarrow$ 自动切分与 embedding $\rightarrow$ 写入 Chroma $\rightarrow$ 立即参与 RAG 检索”这一整套流程已经打通。与之前“仅复制文件、不更新数据库”的状态相比，这个实验案例更能证明当前前端已经具备可验证的在线扩库能力。



图 8: 实验案例一：针对“测试文档 1.txt”提问的验证截图。左侧可见文件上传记录，主界面可见针对新内容的问题与最终回答。



图 9: 实验案例二：对应的检索证据截图。清晰展示了检索到了“测试文档 1.txt”的相关切片作为上下文。

8 问题、排查与改进

8.1 实验过程中遇到的问题

本次推进过程中，遇到的问题主要集中在四个方面。首先是环境依赖不完整，初次运行实验脚本时环境中缺少 `python-dotenv`，导致程序尚未进入 DeepSeek 调用阶段就终止，后续我通过安装 `requirements.txt` 中的全部依赖解决了这一问题，并将 `pytest` 纳入依赖列表，以保证测试环境可复现。其次，方鹭洋的交接文档中曾记录 `AuthenticationError` 是主要阻塞，但在补齐依赖并复跑后，三模式均可正常调用，因此在当前环境下更真实的问题并不是 API 不可用，而是前置依赖未安装完整。再次，模型有时会输出带“回答：”“来源：无”的包装式拒答，为了统一实验口径，我增加了归一化逻辑，将其统一收敛为“根据现有资料无法回答”。最后，当前最稳定命中的文档几乎都来自“大学生心理咨询”主题，这说明现有 OCR 文本、chunk 质量或向量索引构建策略仍有改进空间。

8.2 后续改进方向

结合本次实验结果，我认为后续的优化重点应放在知识库清洗与重新建库上，尤其需要提升博物馆类问题的检索召回率。此外，还可以在检索层引入更多评估指标，例如命中率、平均分数和分位数统计，用更量化的方式观察不同设置下的检索表现。在工程层面，我也可以补充针对 `teammate_chroma` 后端的单元测试，并尝试混合检索或重排序策略，以提升跨主题问答的稳定性。如果时间允许，还可以继续扩展多轮对话、缓存机制等附加功能，以争取额外加分。

9 结论与心得

到目前为止，我们已经完成了一个完整的 RAG 问答闭环。系统能够调用 DeepSeek 完成受约束的答案生成，能够接入方鹭洋构建的 Chroma 向量库并返回标准化检索结果，能够完成三模式对比实验并输出可追溯结果文件，也提供了可交互的 Streamlit 前端，整体上已经满足课程作业对“可运行、可比较、可展示”的要求。

从我的实验结果看，RAG 相比纯 LLM 的优势主要体现在两个方面。其一，当检索到有效证据时，RAG 可以把检索片段压缩成更简洁、更可信的回答；其二，当检索证据不足或明显偏离问题时，RAG 可以通过拒答机制有效减少幻觉。也正因为如此，RAG 的价值并不只是“增强回答能力”，更重要的是“给回答增加边界感”。

与此同时，这次实验也让我看到 RAG 系统的一个现实问题：生成质量往往不是瓶颈，真正决定上限的是知识库质量、切分质量和检索质量。如果检索阶段没有把相关证据找出来，再强的大模型也很难稳定给出正确答案。因此，未来若继续优化，我会优先把精力放在“知识库质量与检索效果”上，而不是单纯更换更强模型。

参考资料

参考文献

[1] Pinecone. Retrieval-Augmented Generation (RAG). https://www.pinecone.io/learn/retrieval-augmented-generation/

[2] LangChain Documentation. RAG Tutorial. https://python.langchain.com/docs/tutorials/rag/

[3] Chroma Documentation. https://docs.trychroma.com/

[4] Sentence-Transformers Documentation. https://www.sbert.net/

[5] DeepSeek API Documentation. https://platform.deepseek.com/api-docs/